

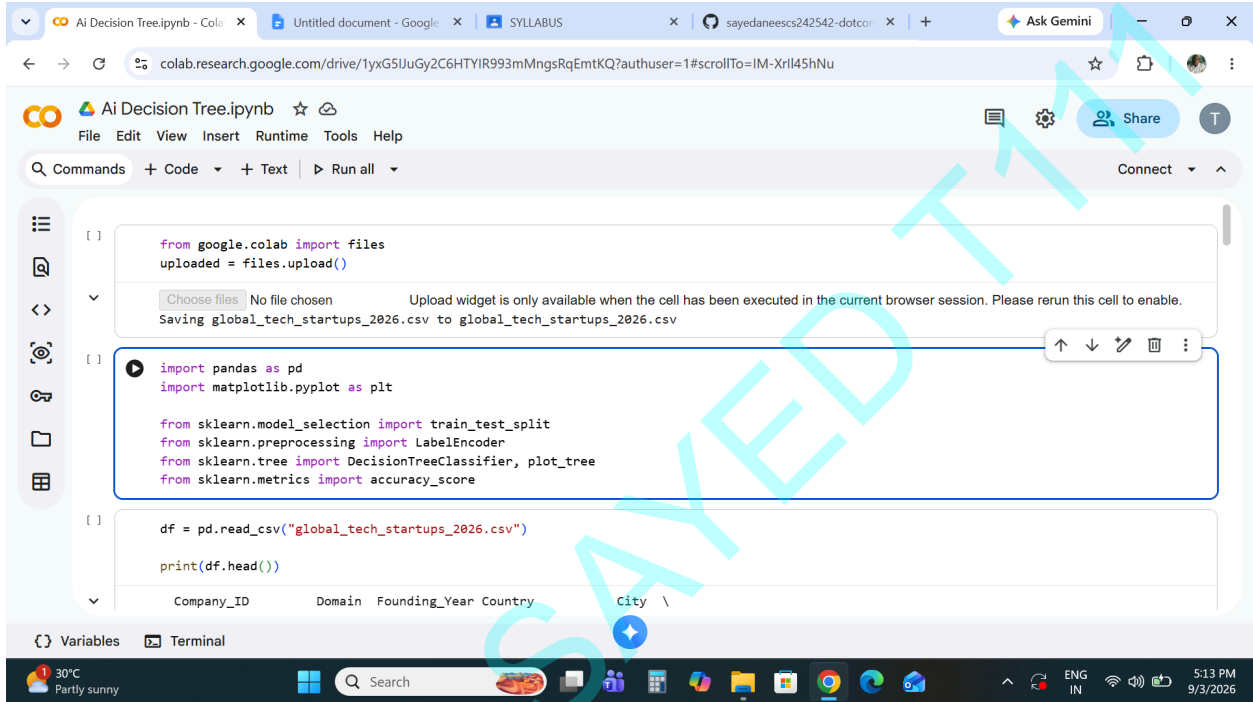
# SHETH MVLU COLLEGE

## SUBJECT:- AI

### Practical 4

AIM:- Decision Tree Learning

- Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.
- Evaluate the accuracy and effectiveness of the decision tree on test data.
- Visualize and interpret the generated decision tree.



The screenshot shows the Google Colab interface with the following code in the first cell:

```
from google.colab import files
uploaded = files.upload()

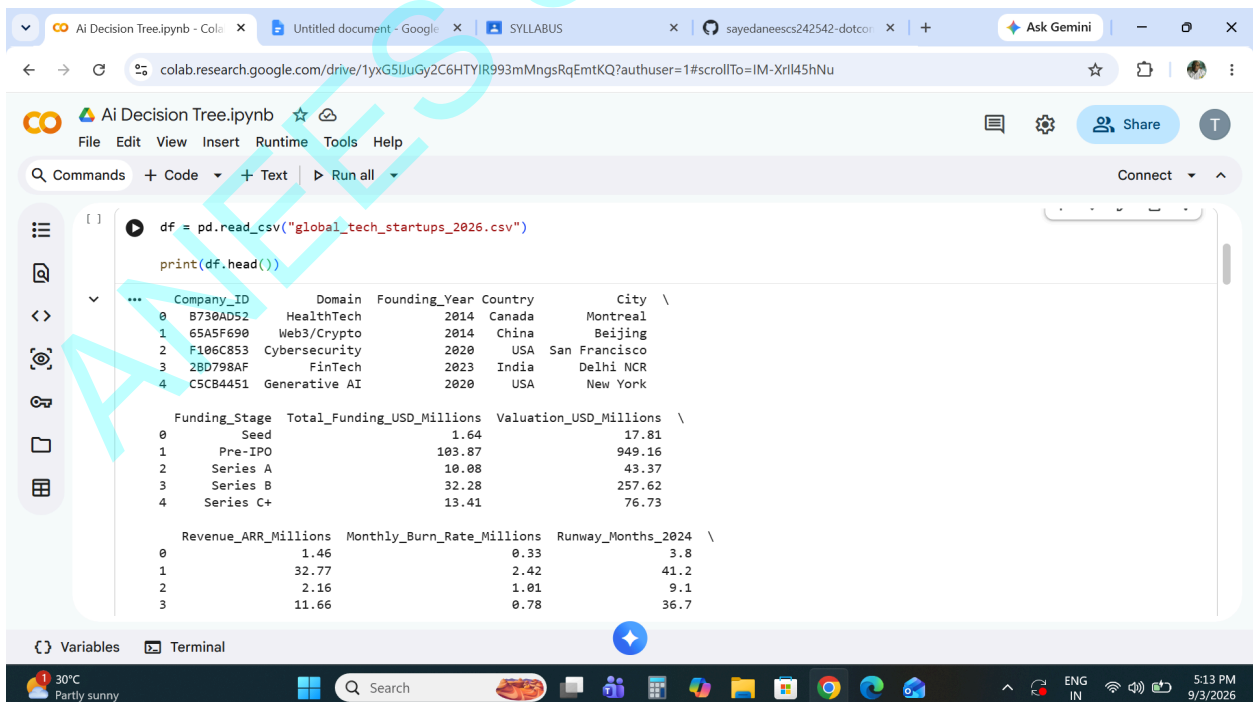
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

df = pd.read_csv("global_tech_startups_2026.csv")
print(df.head())
```

The second cell shows the output of the first cell, displaying the first five rows of the CSV file:

|   | Company_ID | Domain        | Founding_Year | Country | City          |
|---|------------|---------------|---------------|---------|---------------|
| 0 | B730AD52   | HealthTech    | 2014          | Canada  | Montreal      |
| 1 | 65A5F690   | Web3/Crypto   | 2014          | China   | Beijing       |
| 2 | F106C853   | Cybersecurity | 2020          | USA     | San Francisco |
| 3 | 2BD798AF   | FinTech       | 2023          | India   | Delhi NCR     |
| 4 | C5CB4451   | Generative AI | 2020          | USA     | New York      |



The screenshot shows the Google Colab interface with the following code in the second cell:

```
df = pd.read_csv("global_tech_startups_2026.csv")
print(df.head())
```

The output of the second cell shows the first five rows of the CSV file, including columns for Funding\_Stage, Total\_Funding\_USD\_Millions, Valuation\_USD\_Millions, Revenue\_ARR\_Millions, Monthly\_Burn\_Rate\_Millions, and Runway\_Months\_2024:

|   | Company_ID | Domain        | Founding_Year | Country | City          | Funding_Stage | Total_Funding_USD_Millions | Valuation_USD_Millions | Revenue_ARR_Millions | Monthly_Burn_Rate_Millions | Runway_Months_2024 |
|---|------------|---------------|---------------|---------|---------------|---------------|----------------------------|------------------------|----------------------|----------------------------|--------------------|
| 0 | B730AD52   | HealthTech    | 2014          | Canada  | Montreal      | Seed          | 1.64                       | 17.81                  | 1.46                 | 0.33                       | 3.8                |
| 1 | 65A5F690   | Web3/Crypto   | 2014          | China   | Beijing       | Pre-IPO       | 103.87                     | 949.16                 | 32.77                | 2.42                       | 41.2               |
| 2 | F106C853   | Cybersecurity | 2020          | USA     | San Francisco | Series A      | 10.08                      | 43.37                  | 2.16                 | 1.01                       | 9.1                |
| 3 | 2BD798AF   | FinTech       | 2023          | India   | Delhi NCR     | Series B      | 32.28                      | 257.62                 | 11.66                | 0.78                       | 36.7               |
| 4 | C5CB4451   | Generative AI | 2020          | USA     | New York      | Series C+     | 13.41                      | 76.73                  |                      |                            |                    |

ANEES SAYED T111

# SHETH MVLU COLLEGE

## SUBJECT:- AI

The top screenshot shows the initial data loading in a Google Colab notebook. The data is loaded from a CSV file and previewed in a table format. The table has columns for various financial and operational metrics, and a final column for 'Acquisition\_Status'.

|   | Revenue_ARR_Millions | Monthly_Burn_Rate_Millions | Runway_Months_2024 | Acquisition_Status |
|---|----------------------|----------------------------|--------------------|--------------------|
| 0 | 1.46                 | 0.33                       | 3.8                | Independent        |
| 1 | 32.77                | 2.42                       | 41.2               | Closed             |
| 2 | 2.16                 | 1.01                       | 9.1                | Independent        |
| 3 | 11.66                | 0.78                       | 36.7               | Independent        |
| 4 | 2.57                 | 1.12                       | 11.0               | Closed             |

The bottom screenshot shows the data preprocessing code and its output. The code loads the data into a DataFrame, identifies the features and target variable, and uses a LabelEncoder to convert the categorical 'Acquisition\_Status' into numerical values.

```
[1] X = df[["Total_Funding_USD_Millions",
        "Valuation_USD_Millions",
        "Revenue_ARR_Millions",
        "Runway_Months_2024",
        "Current_Headcount_2026"]]

y = df["Acquisition_Status"]

print("Features:")
print(X.columns)

... Features:
Index(['Total_Funding_USD_Millions', 'Valuation_USD_Millions',
      'Revenue_ARR_Millions', 'Runway_Months_2024', 'Current_Headcount_2026'],
      dtype='object')

[1] le = LabelEncoder()

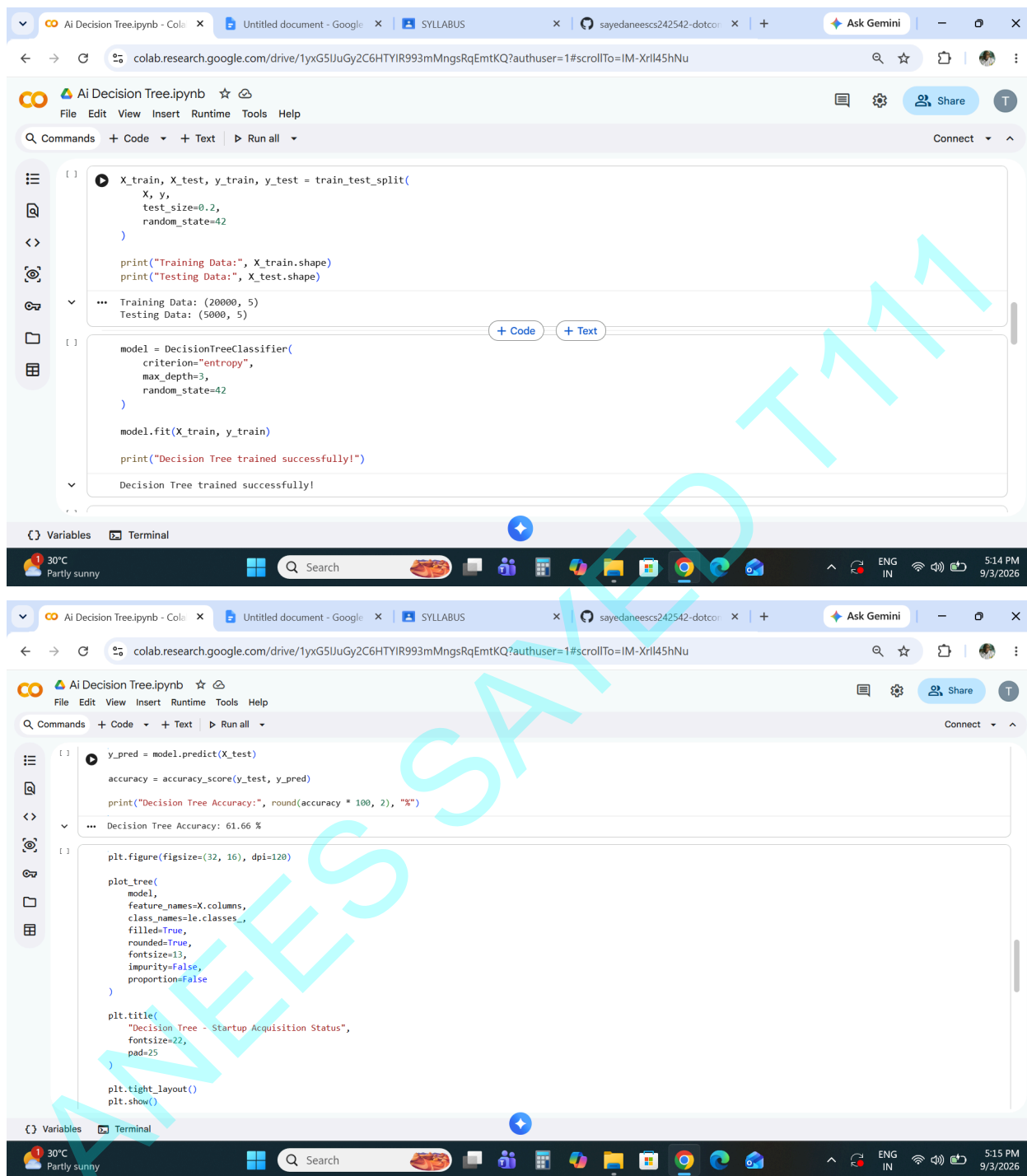
y = le.fit_transform(y)

print("Classes:", le.classes_)

... Classes: ['Acquired' 'Acquired (Fire Sale)' 'Closed' 'IPO' 'Independent']
```

# SHETH MVLU COLLEGE

## SUBJECT:- AI



The image displays two screenshots of a Google Colab notebook titled "AI Decision Tree.ipynb". The notebook is open in a web browser, showing the code editor and the output area.

**First Screenshot:**

```
[1]: X_train, X_test, y_train, y_test = train_test_split(
      X, y,
      test_size=0.2,
      random_state=42
    )

    print("Training Data:", X_train.shape)
    print("Testing Data:", X_test.shape)

    ... Training Data: (20000, 5)
    Testing Data: (5000, 5)

[2]: model = DecisionTreeClassifier(
      criterion="entropy",
      max_depth=3,
      random_state=42
    )

    model.fit(X_train, y_train)

    print("Decision Tree trained successfully!")

    ... Decision Tree trained successfully!
```

**Second Screenshot:**

```
[3]: y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    print("Decision Tree Accuracy:", round(accuracy * 100, 2), "%")

    ... Decision Tree Accuracy: 61.66 %

[4]: plt.figure(figsize=(32, 16), dpi=120)

    plot_tree(
        model,
        feature_names=X.columns,
        class_names=le.classes_,
        filled=True,
        rounded=True,
        fontsize=13,
        impurity=False,
        proportion=False
    )

    plt.title(
        "Decision Tree - Startup Acquisition Status",
        fontsize=22,
        pad=25
    )

    plt.tight_layout()
    plt.show()
```

# SHETH MVLU COLLEGE

## SUBJECT:- AI

Decision Tree - Startup Acquisition Status

